

Лабораторна робота №6: Архітектурний аудит та багаторівнева оцінка часових і фінансових ризиків програмних проєктів

Мета роботи: Набуття практичних навичок проведення комплексного технічного аудиту програмних систем на основі об'єктно-орієнтованих метрик (Lorenz & Kidd, CK, MOOD) та опанування методологій багаторівневого прогнозування трудомісткості (UCP) і часових ризиків (PERT) для прийняття стратегічних управлінських рішень щодо рефакторингу, масштабування та фінансового планування ІТ-проєктів.

Програмна частина завдання: Розробка системи автоматизованого архітектурного аудиту та оцінки ризиків (Architecture Metrics Lab)

Контекст (Кейс):

Керівництво компанії «ElectroSoftware» вирішило автоматизувати процес прийняття рішень щодо рефакторингу. Ваше завдання — розробити програмний продукт (Desktop або Web-застосунок) з графічним інтерфейсом (GUI), який дозволяє проводити комплексний аудит архітектури та розрахунок бюджету проєкту на основі введених параметрів.

Технічні вимоги до функціоналу:

1) Модуль архітектурної діагностики (Завдання 1 & 2):

- Аналізатор ієрархії: Інтерфейс для введення чи імпортування параметрів батьківських та дочірніх класів (NM_base, NOO, NOA, L). Програма повинна автоматично розраховувати Індекс спеціалізації (SI) та виводити статус: «Stable» або «Aggressive Inheritance».
- Dashboard метрик CK: Візуалізація показників WMC, DIT, CBO, LCOM, RFC. Система має підсвічувати «червоним» ті значення, що перевищують встановлені пороги (наприклад, WMC>40).
- MOOD Analyzer: Калькулятор для розрахунку відсоткових показників AHF, MHF, COF, POF на основі загальної кількості методів та атрибутів у проєкті.

2) Модуль фінансово-часового планування (Завдання 3):

- UCP Calculator: Інтерактивна форма для введення кількості Акторів та Use Cases різних рівнів складності. Автоматичний розрахунок людино-годин із врахуванням факторів TCF та ECF.
- PERT Risk Engine: Модуль для введення трьох часових оцінок (О, М, Р). Програма має будувати графік або шкалу ризику, розраховувати очікуваний час (Е) та «безпечний дедлайн» ($E + 2SD$).
- Commercial Proposal Generator: Автоматична генерація підсумкової таблиці з бюджетом розробки, ставкою \$35/год та розрахованим фінансовим резервом на ризики.

3) Вимоги до GUI (Інтерфейсу користувача):

- Панель управління (Sidebar): Навігація між аудитом архітектури та розрахунком бюджету.
- Візуалізація ризиків: Використання кольорових індикаторів (Зелений/Жовтий/Червоний) для швидкої оцінки стану проєкту.

Очікуваний результат:

- Працюючий програмний код (наприклад, на Python/PyQt/Streamlit, C#/WPF або JS/React), який демонструє автоматизований розрахунок для обраного студентом індивідуального варіанту.

ПРАКТИЧНА ЧАСТИНА

Завдання 1: Архітектурний аудит та оцінка стійкості ОО-систем

Контекст (Кейс): Ви працюєте в команді, що розробляє деякий продукт. Замовник хоче додати новий модуль. Ваше завдання — провести аудит поточної архітектури, щоб зрозуміти, чи витримає вона розширення, або ж потребує негайного рефакторингу.

Крок 1: Концептуальне проектування та визначення масштабу системи

Мета: Визначити межі програмного проєкту, його ключові компоненти та функціональні сценарії для подальшого розрахунку архітектурних метрик за методикою Lorenz & Kidd.

Тут студент має виступити в ролі Lead Architect, який описує концепцію системи перед початком детального аудиту.

Ваше завдання:

- 1) Об'єкт дослідження: Використовуйте назву та опис системи з вашого індивідуального варіанту (див. таблицю варіантів).
- 2) Формування показників (Проектний рівень):
 - NSS (Number of Scenario Scripts): Опишіть 3 ключові сценарії (простий, середній, складний) використання системи. Наприклад: "Авторизація користувача", "Генерація фінансового звіту", "Обробка критичного сповіщення від датчика".
 - NKC (Number of Key Classes): Визначте та випишіть назви 5 ключових класів, які є критичними для бізнес-логіки вашої системи (наприклад: User, Transaction, DatabaseConnector тощо).
 - NSU (Number of Subsystems): Виділіть 2-3 логічні підсистеми, з яких складається ваш проєкт (наприклад: BillingModule, AuthService, DataVisualization).
- 3) Обґрунтування масштабу: Напишіть коротке резюме (2-3 речення): чи вважаєте ви систему "масштабною" на основі кількості виділених підсистем (NSU) та сценаріїв (NSS)?
- 4) Візуалізація (Опційно): Намалюйте просту діаграму зв'язків між обраними підсистемами (можна у вигляді блоків).

Приклад виконання Кроку 1 (Сценарій: Система «Smart Parking Solution»)

Об'єкт дослідження: Автоматизована система управління багаторівневим паркінгом (контроль в'їзду, оплата, моніторинг вільних місць).

1. Сценарії використання (NSS — Number of Scenario Scripts):
 - Сценарій 1: «В'їзд автомобіля» (Зчитування номера → Перевірка наявності вільних місць → Відкриття шлагбаума → Друк талона).
 - Сценарій 2: «Оплата послуг» (Сканування талона в терміналі → Розрахунок вартості за часом → Прийом оплати → Оновлення статусу талона).
 - Сценарій 3: «Нічний звіт для адміністратора» (Генерація статистики завантаженості паркінгу за добу та фінансовий підсумок).
2. Ключові класи (NKC — Number of Key Classes):
 - Vehicle (Зберігає дані про авто: номер, тип, статус).
 - ParkingSlot (Керує станом конкретного місця: вільне/зайняте/бронь).
 - PaymentTerminal (Відповідає за логіку розрахунків та транзакції).
 - BarrierController (Керує апаратним забезпеченням: шлагбаумами та датчиками).
 - AdminReportGenerator (Клас для агрегації даних та виведення статистики).
3. Підсистеми (NSU — Number of Subsystems):
 - Subsystem A: AccessControl (Керування в'їздом/виїздом, розпізнавання номерів).
 - Subsystem B: BillingSystem (Тарифікація, обробка платежів, податкова звітність).
 - Subsystem C: MonitoringHub (Інтерфейс для охорони, карта паркінгу в реальному часі).

4. Обґрунтування масштабу (Management Summary):

Система має середній рівень масштабності, оскільки містить чітко розділені підсистеми з різним рівнем відповідальності (Hardware-контроль та FinTech-логіка). Кількість ключових класів (5) та сценаріїв (3) дозволяє провести глибокий аудит без надмірного роздуття документації.

Перелік варіантів для Завдання 1 (Крок 1)

№	Назва системи	Опис бізнес-кейсу
1	SmartClinic	Система управління приватною клінікою (запис, медкартки, рахунки).
2	AeroCheck	Система реєстрації пасажирів та контролю багажу в аеропорту.
3	EcoLogistics	Логістична платформа для відстеження електрофур та оптимізації маршрутів.
4	FoodDash Admin	Бекенд-система для агрегатора доставки їжі (кур'єри, ресторани, замовлення).
5	EduPortal	Платформа дистанційного навчання (курси, тести, сертифікати, успішність).
6	BankGuard	Система моніторингу підозрілих транзакцій та антифрод-аналізу.
7	FitTrack Pro	Екосистема для фітнес-центрів (абонементи, тренування, IoT-датчики).
8	AutoPart Finder	Маркетплейс вживаних автозапчастин з інтеграцією складів СТО.
9	GreenEnergy Grid	Система управління сонячними панелями та продажу енергії в мережу.
10	CinemaBooking	Сервіс продажу квитків у кінотеатри з вибором місць та лояльністю.
11	PetHotel	Система бронювання місць у готелях для тварин з фотозвітами.
12	Warehouse360	Автоматизована система складського обліку (QR-сканування, інвентаризація).
13	CryptoWallet	Гаманець для зберігання та обміну криптовалют з підтримкою API бірж.
14	RealEstate Scan	Агрегатор нерухомості з фільтрами, картами та оцінкою вартості.
15	SmartHome Hub	Центральний сервер управління розумним будинком (світло, безпека, клімат).
16	EventMaster	Система організації конференцій (спікери, розклад, нетворкінг).
17	SteamCloud Shop	Магазин цифрових ігор з системою досягнень та відгуків.
18	LawCase Manager	Система для юристів (документообіг, судові справи, тайм-трекінг).
19	TaxiRide Core	Диспетчерська служба таксі (розподіл замовлень, тарифікація, рейтинг).
20	LibraryNext	Цифрова бібліотека з системою оренди книг та доступом до PDF-архівів.

Крок 2: Аналіз ієрархії та операційної складності (Lorenz & Kidd: NM, NV, NOO, SI)

Мета: Оцінити рівень взаємодії між батьківськими та дочірніми класами, а також визначити обсяг роботи, необхідний для реалізації специфічної логіки нащадків. Також провести оцінку якості проектування ієрархії класів та виявити ризики «агресивного успадкування», що ускладнює підтримку коду.

Ваше завдання:

1) Вихідні дані: Використовуйте пару класів (Батьківський та Дочірній) та їхні кількісні характеристики зі свого індивідуального варіанту (див. таблицю варіантів).

2) Деталізація методів (Операцій):

- NM_{base} (Methods in Base Class): Кількість методів, які вже існують у «батька».
- NOO (Number of Overridden Operations): Кількість методів, які ви переписуєте (змінюєте логіку) у дочірньому класі.
- NOA (Number of Added Operations): Кількість нових методів, які з'являються лише у дочірньому класі.

3) Аналіз атрибутів (NV): Випишіть кількість змінних (полів) класу (NV), які використовуються для збереження стану об'єкта-нащадка.

4) Розрахунок загального обсягу методів: Обчисліть загальну кількість методів у дочірньому класі за формулою:

$$M_{total} = (NM_{base} - NOO) + NOO + NOA$$

5) Обчисліть Індекс спеціалізації (SI) за формулою:

$$SI = \frac{NOO \times L}{M_{total}}$$

6) Аналіз стабільності:

- Якщо $SI \leq 0.5$: Успадкування є коректним, нащадок доповнює батька, не ламаючи його логіку.

- Якщо $SI > 1.0$: Нащадок занадто сильно перевизначає батьківські методи. Це ознака того, що успадкування використано штучно, і класи варто зробити незалежними.

Варіанти для Завдання 1 (Крок 2): Аналіз ієрархії

№	Батьківський клас (NM base)	Дочірній клас	Перевизначені (NOO)	Додані (NOA)	Атрибути (NV)	Рівень (L)
1	MedicalUser (8)	Surgeon	4	2	12	1
2	AirportAsset (10)	BaggageScanner	3	5	8	1
3	BaseVehicle (6)	ElectricTruck	4	3	10	1
4	OrderProcessor (12)	ExpressDelivery	2	4	6	1
5	CourseContent (7)	InteractiveQuiz	5	1	9	1
6	Transaction (15)	HighRiskTransfer	8	2	14	1
7	FitnessService (9)	PersonalTraining	2	6	5	1
8	CatalogItem (11)	RarePart	3	2	7	1
9	EnergySource (5)	SolarPanelArray	4	4	15	1
10	Booking (8)	VIP_Reservation	6	3	11	1
11	AnimalProfile (10)	ExoticPet	2	5	8	1
12	StorageUnit (12)	RefrigeratedZone	7	1	13	1
13	CryptoAsset (7)	StableCoin	3	4	9	1
14	Property (14)	CommercialLease	5	5	20	1
15	SmartDevice (9)	SecurityCamera	4	8	12	1
16	EventParticipant (6)	KeynoteSpeaker	2	3	5	1
17	DigitalProduct (11)	DLC_Content	9	2	6	1
18	LegalDocument (13)	CourtAppeal	6	4	18	1
19	TaxiService (8)	PremiumRide	3	2	7	1
20	ArchiveItem (10)	RareManuscript	5	6	14	1

Крок 3: Глибокий аудит архітектурної складності (Метрики СК)

Мета: Провести діагностику ключового керуючого класу обраної системи (Controller або Manager) за методикою Чидамбера та Кемерера для визначення ризиків підтримки, тестування та стабільності коду.

Ваше завдання:

1) Вихідні дані: Використовуйте кількісні показники WMC, DIT, CBO, LCOM та RFC для головного класу вашої системи, наведені у вашому індивідуальному варіанті.

2) Параметри аналізу:

- WMC (Weighted Methods per Class): Оцініть складність логіки класу. Чи не є він перевантаженим?
- DIT (Depth of Inheritance Tree): Перевірте глибину вкладеності. Чи легко відстежити помилку в ієрархії?
- CBO (Coupling Between Objects): Визначте рівень залежності від інших об'єктів. Чи не «впаде» ваш клас при зміні суміжного модуля?
- LCOM (Lack of Cohesion in Methods): Оцініть цілісність класу. Чи методи працюють разом, чи клас — це випадковий набір функцій?
- RFC (Response For a Class): Проаналізуйте «радіус викликів». Скільки тест-кейсів знадобиться для повного покриття?

3) Аналітичний розрахунок:

- Порівняйте отримані цифри з еталонними порогоми (використовуйте наведений Гайд)
- Визначте «червоні зони» вашого класу — показники, що перевищують норму.

4) Формування «Managerial Report» (Управлінського звіту):

Напишіть короткий професійний висновок (4-6 речень), у якому дайте відповіді на питання:

- Чи є даний клас «God Object» (занадто складним) і як це вплине на вартість розробки?
- Наскільки клас ізольований від інших? Чи не призведе зміна в суміжному модулі до каскадної помилки у вашому класі (оцінка CBO та RFC)?

- Який прогноз щодо тестування: чи легко буде покрити цей клас Unit-тестами, чи він потребує складної інфраструктури?

5) Ваш вердикт: Дайте фінальну рекомендацію керівництву: «Залишити як є», «Провести частковий рефакторинг» або «Негайно переписати архітектуру».

Варіанти для Завдання 1 (Крок 3): Метрики СК (Chidamber & Kemerer)

№	Головний клас (Об'єкт аналізу)	WMC	DIT	CBO	LCOM	RFC
1	ClinicManager	28	3	14	0.82	45
2	FlightController	35	2	18	0.45	60
3	RouteOptimizer	22	4	9	0.20	38
4	DeliveryDispatcher	42	3	22	0.91	75
5	ExamEngine	18	2	7	0.33	25
6	FraudDetector	55	5	12	0.60	90
7	TrainingScheduler	20	3	11	0.77	30
8	InventorySync	26	2	15	0.50	42
9	GridBalancer	48	4	19	0.88	82
10	CinemaReservation	15	1	6	0.15	22
11	PetBookingSystem	24	3	13	0.65	40
12	WarehouseRobotAI	38	6	25	0.95	110
13	WalletSecurity	12	2	5	0.10	18
14	MarketAnalyzer	31	4	16	0.70	55
15	SmartHomeCore	45	5	21	0.85	95
16	EventCoordinator	19	2	10	0.40	33
17	GameEngine	60	7	30	0.98	150
18	CaseManager	27	3	12	0.55	48
19	TaxiFleetControl	33	4	17	0.78	65
20	DigitalArchive	14	2	8	0.25	20

Гайд з інтерпретації метрик для Управлінського звіту (Management Summary)

Як майбутній Project Manager або Lead Architect, ти маєш оцінити «здоров'я» коду за наступними критичними показниками:

1) WMC (Weighted Methods per Class) — Складність класу

- Що це: Сумарна складність усіх методів у класі.
- Поріг: Норма до 20. Критично — більше 40.
- Вердикт для звіту:
 - Нижче норми: Клас фокусований, його легко тестувати та підтримувати.
 - Вище норми: Клас перетворився на «God Object» (Божественний об'єкт), який робить забагато справ одночасно. Це призводить до того, що будь-яка зміна в логіці може зламати суміжні функції.
 - Рішення: Рефакторинг шляхом розділення на декілька менших класів.

2) DIT (Depth of Inheritance Tree) — Глибина успадкування

- Що це: Максимальний шлях від кореня ієрархії до даного класу.
- Поріг: Норма 1–4. Критично — більше 5.
- Вердикт для звіту:
 - Нижче норми: Проста та зрозуміла структура.
 - Вище норми: Занадто глибоке успадкування. Помилка в базовому класі («дереві») прокочується через усі гілки. Розробнику важко зрозуміти, звідки прийшов той чи інший метод.
 - Рішення: Використовувати композицію замість успадкування.

3) CBO (Coupling Between Objects) — Зв'язність між об'єктами

- Що це: Кількість інших класів, з якими пов'язаний даний клас.
- Поріг: Норма до 12. Критично — більше 20.
- Вердикт для звіту:

- Нижче норми: Клас автономний. Його можна легко перевикористати в іншому проєкті.
- Вище норми: Клас занадто залежний. Якщо ви зміните один із 20 суміжних класів, цей клас «впаде». Це створює ефект «крихкої архітектури».
- Рішення: Зменшення прямих залежностей через інтерфейси (Dependency Injection).

4) LCOM (Lack of Cohesion in Methods) — Незв'язність методів

- Що це: Показник того, наскільки методи класу працюють зі спільними даними (полями).
- Поріг: Чим ближче до 0 — тим краще. Ближче до 1.0 — критично.
- Вердикт для звіту:
 - Низький LCOM: Клас є цілісним механізмом. Всі методи працюють на одну мету.
 - Високий LCOM: Клас — це просто «смітник» випадкових функцій, які ніяк не пов'язані між собою. Це ускладнює розуміння коду.
 - Рішення: Розбити клас на логічно пов'язані модулі.

5) RFC (Response For a Class) — Відгук класу

- Що це: Кількість методів (своїх + чужих), які можуть бути викликані у відповідь на запит.
- Поріг: Норма до 50. Критично — більше 100.
- Вердикт для звіту:
 - Нижче норми: Клас має чіткий та обмежений радіус дії.
 - Вище норми: Клас ініціює занадто складні ланцюжки викликів. Це кошмар для тестувальника, оскільки покрити тестами такий «вибух» викликів майже неможливо.
 - Рішення: Спрощення логіки взаємодії, зменшення кількості зовнішніх викликів.

Загальний висновок для РМ:

Якщо у вашому варіанті більше 2-х показників перевищують норму — проєкт має високий Технічний борг. Це означає, що вартість підтримки (Maintenance Cost) буде зростати з кожним місяцем, а імовірність появи багів при кожному релізі є критичною.

Крок 4: Аналіз зв'язності (Coupling)

Мета: На прикладі взаємодії об'єктів вашого варіанту обґрунтувати переваги слабкої зв'язності (Loose Coupling) та її вплив на гнучкість системи.

Ваше завдання:

- 1) Порівняльний аналіз: Розгляньте два сценарії взаємодії між Класом А та Класом Б (див. свій варіант у таблиці).
- 2) Обґрунтування вибору: Поясніть, чому Сценарій Б відповідає принципу Loose Coupling. Чому передача мінімального набору даних (наприклад, тільки ID та суми) є безпечнішою та кращою за передачу всього об'єкта (Сценарій А)?
- 3) Оцінка Fan-out: Як перехід до Сценарію Б впливає на показник Fan-out Класу А? (Підказка: чи стає Клас А менш залежним від структури Класу Б?).
- 4) Бізнес-кейс "Заміна модуля": Уявіть, що вам потрібно змінити технологію в Класі Б (наприклад, змінити сервіс друку чи платіжну систему). Чому в Сценарії Б це займе лише 1 годину, а в Сценарії А може призвести до переписування всієї системи?

Варіанти для Завдання 1 (Крок 4): Аналіз зв'язності (Coupling)

Кожен студент аналізує взаємодію Клас-відправник → Клас-отримувач.

№	Система	Клас А (Відправник)	Клас Б (Отримувач)	Сценарій А (Tight - що передаємо)	Сценарій Б (Loose - що передаємо)
1	SmartClinic	Doctor	E_Recipe	Весь об'єкт Patient та History	patientID та drugCode
2	AeroCheck	GateControl	BoardingPass	Об'єкт Passenger зі всіма візами	ticketID та gateStatus
3	EcoLogistics	Route	GPS_Tracker	Весь список Waypoints та Cargo	currentCoords та speed
4	FoodDash	Courier	PayoutSystem	Об'єкт Order з переліком страв	orderId та deliveryFee
5	EduPortal	Student	Certificate	Весь GradeBook за всі роки	courseID та finalScore
6	BankGuard	Transaction	FraudEngine	Об'єкт Cardholder з адресою	amount та merchantCategory
7	FitTrack	Workout	CalorieCalc	Об'єкт User з антропометрією	activityType та duration
8	AutoPart	Store	Shipping	Об'єкт Customer з кошиком	weight та zipCode
9	GreenGrid	Sensor	MainInverter	Весь лог VoltageHistory	currentVoltage
10	Cinema	Ticket	Printer	Об'єкт Movie з описом та акторами	seat та bookingTime
11	PetHotel	Booking	VaccineCheck	Весь об'єкт Pet з родоводом	petType та vaxDate
12	Warehouse	Robot	Inventory	Весь об'єкт Shelf з координатами	skuCode та actionType
13	Crypto	Trade	TaxCalculator	Весь WalletHistory	profitAmount та assetType
14	RealEstate	Agent	AdPublisher	Об'єкт Property з кресленнями	price та squareMeters
15	SmartHome	Light	ElectricityMeter	Об'єкт Room зі списком меблів	devicePower та state
16	EventMaster	Guest	BadgeGenerator	Весь профіль LinkedIn	fullName та roleType
17	SteamCloud	Achievement	UserRank	Весь об'єкт GameStats	xpPoints та isTrophy
18	LawCase	Judge	CourtVerdict	Весь об'єкт Evidence (докази)	caseID та verdictStatus
19	TaxiRide	Driver	RatingSystem	Об'єкт Trip з маршрутом на карті	rating та tripDuration
20	Library	Reader	FineCalculator	Весь список BorrowedBooks	overdueDays та bookTier

Крок 5: Управлінський звіт та стратегія рефакторингу

Мета: Синтезувати результати всіх попередніх етапів аудиту (Lorenz & Kidd, СК) для прийняття остаточного рішення щодо готовності архітектури до релізу або подальшого розширення.

Тут студент виступає вже не як математик, а як Chief Technology Officer (CTO), який ставить крапку в аудиті архітектури.

Ваше завдання:

На основі розрахованого вами показника SI (Індекс спеціалізації) та отриманих значень CBO, LCOM, WMC, сформулюйте розширений аналітичний висновок (Managerial Report). У звіті обов'язково дайте відповіді на наступні питання:

1) Готовність до впровадження: Чи готова архітектура системи до інтеграції нових модулів прямо зараз? Підказка: Якщо WMC>40 або CBO>20, впровадження нового коду може призвести до "ефекту доміно" (поломки старих функцій).

2) Діагностика ризиків: Які саме метрики сигналізують про найбільші технічні проблеми?

- Високий SI (> 0.8): Проблема в ієрархії (нащадки "ламають" логіку батьків).
- Високий CBO (> 15): Проблема в залежностях (система занадто заплутана).

- Високий LCOM (> 0.7): Проблема в логіці класу (він робить забагато несхожих речей).

3) Пріоритетний крок рефакторингу: Яку одну дію ви пропонуєте зробити першою, щоб максимально покращити стан системи? Варіанти:

- Розділити God Object на дрібні класи,
- Впровадити інтерфейси для зменшення зв'язності (CBO),
- Спростити ієрархію успадкування.

Приклад структури звіту: Висновок за результатами аудиту системи "AeroCheck":

Архітектура системи на даному етапі не готова до впровадження модуля міжнародних рейсів. Найбільший ризик становить показник CBO (18), що вказує на критичну залежність головного контролера від підсистем багажу та безпеки. Крім того, SI (0.95) свідчить про те, що дочірні класи фактично ігнорують логіку базових компонентів, що робить успадкування фіктивним.

Першочерговий крок: Рекомендую провести рефакторинг головного класу FlightController шляхом виділення логіки перевірки документів в окремий сервіс. Це знизить показник WMC та дозволить стабілізувати залежності перед релізом.

Критерії оцінювання:

- Точність розрахунків: Коректне використання формул (особливо SI та WMC).
- Глибина аналізу: Здатність пояснити, чому DIT=6 — це небезпечно для бюджету.
- Архітектурне рішення: Обґрунтованість пропозицій щодо декомпозиції класів.

Завдання 2: Стратегічний аудит проєкту за методикою MOOD

Мета роботи: Оцінити якість об'єктно-орієнтованого дизайну на рівні всього проєкту, проаналізувати ефективність інкапсуляції, успадкування та поліморфізму, а також виявити ризики надмірної зв'язності (Coupling).

Контекст (Кейс)

Ви отримали результати автоматизованого сканування коду вашого продукту. Ваше завдання — інтерпретувати метрики MOOD, порівняти їх із цільовими показниками та скласти звіт для СТО про технічний стан системи.

Гайд "Червоних прапорців" для студента: Параметри аналізу (Розуміння показників):

- MHF (Method Hiding Factor): Відсоток прихованих (private/protected) методів. Чи не забагато внутрішньої логіки доступно ззовні?
- AHF (Attribute Hiding Factor): Рівень інкапсуляції даних. Чи захищені поля класу від прямого доступу?
- MIF (Method Inheritance Factor): Частка методів, отриманих через успадкування. Чи не є система занадто «громіздкою» через ієрархію?
- POF (Polymorphism Factor): Рівень використання інтерфейсів та віртуальних методів. Чи легко замінити один модуль іншим без зміни коду?

MHF (Method Hiding Factor) — Інкапсуляція логіки

- Що означає: Який відсоток методів приховано (private/protected) від інших класів.
- Норма: 0.4 – 0.6 (40-60%).
- Для звіту:

- Занизький (< 0.25): «Інтерфейсна надмірність». Система занадто відкрита, будь-який розробник може викликати внутрішні методи, що призведе до непередбачуваних помилок при оновленні версій.
- Зависокий (> 0.8): «Ізоляція». Класи настільки закриті, що їх важко перевикористовувати або тестувати.

АНФ (Attribute Hiding Factor) — Захищеність даних

- Що означає: Наскільки поля (змінні) класу захищені від прямого доступу ззовні.
- Норма: 0.8 – 1.0 (80-100%).
- Для звіту:
 - Занизький (< 0.8): Критичний ризик! Дані (ціни, паролі, статуси) змінюються напряму без перевірок (геттерів/сеттерів). Це «діра» в безпеці та стабільності системи.

МІФ (Method Inheritance Factor) — Ефективність успадкування

- Що означає: Яку частку функціоналу класи отримують від «батьків», а не пишуть з нуля.
- Норма: 0.2 – 0.7 (20-70%).
- Для звіту:
 - Занизький (< 0.2): Код дублюється. Замість того, щоб використати готову логіку, розробники пишуть її щоразу заново.
 - Зависокий (> 0.8): «Крихка ієрархія». Зміна в одному базовому класі може зламати десятки нащадків. Система занадто складна для розуміння.

РОФ (Polymorphism Factor) — Гнучкість (Поліморфізм)

- Що означає: Наскільки часто використовуються інтерфейси та перевизначення методів замість жорстких умов if/else.
- Норма: 0.1 – 0.4 (10-40%).
- Для звіту:
 - Занизький (< 0.1): «Дубовий код». Кожна нова функція (наприклад, новий спосіб оплати) вимагає втручання в уже працюючий код. Це дорого і довго.
 - Зависокий (> 0.5): Зайва складність. Архітектура переобтяжена абстракціями, які не використовуються.

Крок 1: Аналіз інкапсуляції (МНФ та АНФ)

Мета: Самостійно розрахувати показники інкапсуляції проєкту та надати менеджерську оцінку «герметичності» архітектури.

Ваше завдання:

1) Розрахунок: Використовуючи дані свого варіанту, обчисліть:

- МНФ (Коефіцієнт приховування методів).
- АНФ (Коефіцієнт приховування атрибутів).

$$\text{МНФ (Method Hiding Factor)} = 1 - \frac{\text{Кількість публічних методів}}{\text{Загальна кількість методів}}$$

$$\text{АНФ (Attribute Hiding Factor)} = 1 - \frac{\text{Кількість публічних атрибутів}}{\text{Загальна кількість атрибутів}}$$

2) Аналіз результатів:

- Чи відповідає ваш показник АНФ нормі (0.8–1.0)? Якщо він нижчий, поясніть, чому прямий доступ до полів (public attributes) створює ризики для вашого бізнес-кейсу.
- Оцініть МНФ: чи не забагато методів виставлено назовні як "публічні"?

3) Managerial Report: Сформулюйте висновок для розробників:

- Сформулюйте висновок: чи можна назвати таку архітектуру «герметичною» (професійною)?
- Яку рекомендацію ви дасте команді розробників щодо модифікаторів доступу (Private/Public)?

Індивідуальні варіанти для Завдання 2 (Крок 1): Розрахунок MHF та AHF

№	Назва системи	Заг. к-ть методів (M_tot)	Публічні методи (M_pub)	Заг. к-ть атрибутів (A_tot)	Публічні атрибути (A_pub)
1	SmartClinic	120	45	80	4
2	AeroCheck	150	60	100	25
3	EcoLogistics	80	56	60	12
4	FoodDash	200	160	120	60
5	EduPortal	95	40	75	5
6	BankGuard	180	30	110	0
7	FitTrack	70	35	50	10
8	AutoPart	110	77	90	36
9	GreenGrid	130	52	85	8
10	Cinema	65	40	40	6
11	PetHotel	85	38	55	14
12	Warehouse	140	112	105	58
13	CryptoWallet	160	16	95	0
14	RealEstate	105	55	70	18
15	SmartHome	125	45	90	4
16	EventMaster	90	65	65	22
17	SteamCloud	170	80	115	12
18	LawCase	100	35	80	2
19	TaxiRide	115	58	75	11
20	Library	75	50	45	18

Крок 2. Оцінка ефективності успадкування (MIF та AIF)

Мета: Проаналізувати ефективність використання механізму успадкування в архітектурі та виявити ризики «крихкої ієрархії» або надмірного дублювання коду.

Ми переходимо до аналізу ієрархії. Студенти мають розрахувати, яку частку функціоналу класи отримують у «спадок», а не пишуть з нуля. Це показує рівень повторного використання коду та складність архітектурного дерева.

Ваше завдання:

1) Розрахунок: Використовуючи дані свого варіанту, обчисліть:

- MIF (Коефіцієнт успадкування методів).
- AIF (Коефіцієнт успадкування атрибутів).

$$\text{MIF (Method Inheritance Factor)} = \frac{\text{Кількість успадкованих методів}}{\text{Загальна кількість методів у всіх класах}}$$

$$\text{AIF (Attribute Inheritance Factor)} = \frac{\text{Кількість успадкованих атрибутів}}{\text{Загальна кількість атрибутів у всіх класах}}$$

2) Аналітична оцінка (Managerial Report):

- Забагато успадкування (якщо $\text{MIF} > 0.7$). Поясніть чому це може бути проблемою для розробників.
- Замало успадкування (якщо $\text{MIF} < 0.2$). Поясніть чому це може бути проблемою для розробників.

3) Фінальна порада: Який рівень ієрархії ви рекомендуєте (спростити ієрархію чи, навпаки, виділити спільні інтерфейси у батьківські класи)?

Індивідуальні варіанти для Завдання 2 (Крок 2): Розрахунок MIF та AIF

№	Назва системи	Заг. методи (M_tot)	Успадковані методи (M_inh)	Заг. атрибути (A_tot)	Успадковані атрибути (A_inh)
1	SmartClinic	240	110	90	30
2	AeroCheck	310	180	120	75
3	EcoLogistics	180	45	70	15
4	FoodDash	450	95	180	40

5	EduPortal	210	140	85	60
6	BankGuard	380	70	150	25
7	FitTrack	150	90	60	45
8	AutoPart	260	190	110	85
9	GreenGrid	290	120	95	40
10	Cinema	140	30	50	10
11	PetHotel	190	135	75	55
12	Warehouse	330	260	140	115
13	CryptoWallet	220	25	100	10
14	RealEstate	240	115	80	35
15	SmartHome	270	165	110	70
16	EventMaster	195	60	70	20
17	SteamCloud	360	275	145	110
18	LawCase	225	105	90	40
19	TaxiRide	265	140	85	50
20	Library	160	115	65	50

Крок 3. Гнучкість та зв'язність (POF та COF)

Мета: Оцінити адаптивність системи до змін та рівень взаємозалежності модулів, що впливає на стабільність релізів.

Ваше завдання:

1) Розрахунок: Обчисліть показники POF та COF за даними свого варіанту.

$$\text{POF (Polymorphism Factor)} = \frac{\text{Кількість перевизначених методів}}{\text{Максимально можлива кількість перевизначень}}$$

$$\text{COF (Coupling Factor)} = \frac{\text{Кількість фактичних зв'язків між класами}}{\text{Максимально можлива кількість зв'язків}}$$

2) Аналітична оцінка (Managerial Report):

- Оцінка POF (Гнучкість): Якщо $\text{POF} < 0.1$, поясніть, чому додавання нового функціоналу вимагатиме переписування існуючих класів замість використання інтерфейсів?
- Оцінка COF (Зв'язність): Якщо $\text{COF} > 0.15$, поясніть, чому зміна в одному модулі може призвести до поломки всієї системи.

3) Стратегічне рішення: Який показник для вашої системи є більш критичним? Чи пропонуєте ви впровадити Dependency Injection для зменшення COF або Інтерфейси для підвищення POF?

Індивідуальні варіанти для Завдання 2 (Крок 3): Розрахунок POF та COF

№	Назва системи	Перевизначені методи (M _{over})	Макс. можливі (M _{max ov})	Фактичні зв'язки (C _{act})	Макс. можливі (C _{max})
1	SmartClinic	12	80	15	110
2	AeroCheck	25	100	45	110
3	EcoLogistics	8	95	10	90
4	FoodDash	5	120	85	150
5	EduPortal	18	75	12	85
6	BankGuard	35	90	20	140
7	FitTrack	10	65	30	70
8	AutoPart	14	110	55	120
9	GreenGrid	22	85	25	100
10	Cinema	15	70	8	65
11	PetHotel	9	80	35	75
12	Warehouse	4	130	90	140
13	CryptoWallet	40	100	15	130
14	RealEstate	11	85	40	95
15	SmartHome	30	115	22	125
16	EventMaster	13	75	38	80
17	SteamCloud	7	150	75	160
18	LawCase	28	95	18	105
19	TaxiRide	20	110	42	115

20	Library	16	65	28	60
----	---------	----	----	----	----

Крок 4: Підсумковий звіт та архітектурний аудит (MOOD)

Мета: Систематизувати розраховані показники інкапсуляції, успадкування, гнучкості та зв'язності для формування фінального висновку про якість програмного продукту.

Ваше завдання:

Заповнення аудиторської таблиці: На основі результатів Кроків 1–3 заповніть порівняльну таблицю. Примітка: Для ANF, MHF, COF та POF переведіть свої десяткові дроби у відсотки (наприклад, 0.85 → 85%).

Заповніть порівняльну таблицю та надайте рекомендації.

Метрика	Ваш результат	Цільове значення	Стан (OK/Risk/Critical)
ANF	N %	> 95%	
MHF	N %	40-70%	
COF	N %	< 15%	
POF	N %	> 20%	

Визначення статусу (Стан):

- OK: Показник у межах або кращий за цільове значення.
- Risk: Відхилення від норми на 10-20%. Система потребує уваги.
- Critical: Значне відхилення. Архітектура нестабільна, висока імовірність багів при релізі.

Аналітичне резюме (Managerial Report):

Напишіть фінальну рекомендацію для замовника (3-5 речень):

- Який показник є найслабшою ланкою вашої системи?
- Який бізнес-ризик це несе?
- Чи рекомендуєте ви проводити реліз у поточному стані?

Критерії оцінювання:

- Точність розрахунків: Правильне застосування відсоткових співвідношень MOOD.
- Стратегічне мислення: Здатність пояснити, чому високий COF — це «фінансове самогубство» при масштабуванні.
- Якість рекомендацій: Конкретні пропозиції щодо покращення інкапсуляції та поліморфізму.

Завдання 3: Комплексна оцінка трудомісткості та часових ризиків проєкту

Мета роботи: Оволодіти методиками ранньої оцінки (Application Composition), детального розрахунку за сценаріями (UCP) та математичного моделювання ризиків (PERT) на прикладі реального IT-кейсу.

Контекст (Кейс)

Ваша компанія бере участь у тендері на розробку мобільного застосунку для кур'єрів інтернет-магазину. Замовник вимагає не просто «ціну», а обґрунтований розрахунок людино-годин та графік релізів з урахуванням ризиків.

Крок 1. Рання оцінка (Application Composition Model)

Мета: На основі функціонального складу системи розрахувати "чисту" складність проєкту в Object Points та спрогнозувати трудомісткість у людино-місяцях.

Ваше завдання:

Уявіть, що у вас є лише прототип (макети екранів). Оцініть обсяг робіт в Object Points (OP).

1) Розрахунок OP (Object Points): Використовуючи свій варіант, порахуйте загальну кількість точок за формулою:

$$OP = \sum (\text{Кількість елементів} \times \text{Вага складності})$$

Ваги складності (традиційні):

- Екрани (Screens): S (Simple) = 1; M (Medium) = 2; H (Hard) = 3.
- Звіти (Reports): S (Simple) = 2; M (Medium) = 5; H (Hard) = 8.
- Компоненти (3GL): Кожен 3GL модуль (готовий компонент) додає 10 точок.

2) Розрахунок трудомісткості (Effort): Обчисліть кількість людино-місяців (PM) за формулою:

$$PM = \frac{OP \times (100 - \%reuse)}{PROD}$$

Прийміть рівень повторного використання (%reuse) = 20% для всіх варіантів.

3) Managerial Report:

- Скільки розробників потрібно найняти, щоб завершити цей обсяг робіт за 3 календарні місяці?
- Ризик-аналіз: Як низька продуктивність команди (PROD) впливає на фінальну дату релізу у вашому варіанті?

Індивідуальні варіанти для Завдання 3 (Крок 1): Розрахунок Object Points (OP)

№	Назва системи	К-ть екранів (S/M/H)	К-ть звітів (S/M/H)	К-ть 3GL модулів	Продуктивність (PROD)
1	SmartClinic	5 / 3 / 1	2 / 1 / 1	3	12 (Very High)
2	AeroCheck	8 / 4 / 2	4 / 2 / 1	5	7 (Low)
3	EcoLogistics	4 / 2 / 0	2 / 1 / 0	2	10 (High)
4	FoodDash	12 / 6 / 3	6 / 3 / 2	8	5 (Very Low)
5	EduPortal	6 / 3 / 1	3 / 2 / 0	4	12 (Very High)
6	BankGuard	10 / 5 / 4	5 / 4 / 3	10	10 (High)
7	FitTrack	4 / 2 / 1	2 / 1 / 1	2	7 (Low)
8	AutoPart	7 / 4 / 1	4 / 2 / 1	5	10 (High)
9	GreenGrid	9 / 3 / 2	3 / 2 / 2	6	12 (Very High)
10	Cinema	5 / 2 / 1	2 / 1 / 0	3	15 (Extra High)
11	PetHotel	6 / 3 / 0	3 / 2 / 0	3	10 (High)
12	Warehouse	11 / 5 / 3	5 / 3 / 2	9	5 (Very Low)
13	CryptoWallet	8 / 4 / 5	4 / 3 / 4	12	12 (Very High)
14	RealEstate	7 / 3 / 1	3 / 2 / 1	4	10 (High)
15	SmartHome	10 / 6 / 2	5 / 4 / 1	7	10 (High)
16	EventMaster	5 / 4 / 1	2 / 2 / 1	3	7 (Low)
17	SteamCloud	15 / 8 / 4	8 / 5 / 3	15	5 (Very Low)
18	LawCase	6 / 3 / 2	3 / 2 / 1	4	12 (Very High)
19	TaxiRide	9 / 4 / 1	4 / 3 / 1	6	10 (High)
20	Library	4 / 2 / 0	2 / 1 / 0	2	15 (Extra High)

Крок 2. Детальна оцінка за Use Case Points (UCP)

Мета: Провести детальну оцінку обсягу робіт на основі аналізу функціональних прецедентів та акторів системи.

Ваше завдання:

Студенти мають класифікувати дані за такими правилами:

- Актори (UAW): Простий (API/CLI) = 1; Середній (Протоколи TCP/HTTP) = 2; Складний (GUI/Людина) = 3.

- Прецеденти (UUCW): Простий (<3 транзакції) = 5; Середній (4-7 транзакцій) = 10; Складний (>7 транзакцій) = 15.

1) Розрахунок UAW (Unadjusted Actor Weight): Помножте кількість акторів кожного типу на їх вагу (S=1, M=2, H=3) та просумуйте.

2) Розрахунок UUCW (Unadjusted Use Case Weight): Помножте кількість Use Cases кожного типу на їх вагу (S=5, M=10, H=15) та просумуйте.

3) Розрахунок UUCP (Unadjusted Use Case Points): $UUCP = UAW + UUCW$.

4) Фінальний розрахунок UCP: $UCP = UUCP * TCF * ECF$.

5) Managerial Report:

- Прийміть, що на реалізацію 1 UCP команда витрачає 20 людино-годин. Який загальний бюджет часу потрібен для вашого проєкту?
- Ризик: Як високий Технічний фактор (TCF) у вашому варіанті (якщо він > 1.0) впливає на підсумкову складність? Які технічні складнощі це може бути (наприклад, розподіленість системи, вимоги до безпеки)?

Шпаргалка для студента:

- UAW: Відображає "зовнішню" складність (з чим система інтегрується).
- UUCW: Відображає "внутрішню" складність (логіку транзакцій).
- TCF/ECF: Коригують оцінку залежно від досвіду команди та складності середовища.

Індивідуальні варіанти для Завдання 3 (Крок 2): Розрахунок UCP

№	Назва системи	К-ть Акторів (S / M / H)	К-ть Use Cases (S / M / H)	TCF (Техн. фактор)	ECF (Екол. фактор)
1	SmartClinic	2 / 1 / 2	5 / 3 / 1	0.95	0.85
2	AeroCheck	4 / 3 / 1	8 / 4 / 2	1.10	1.05
3	EcoLogistics	1 / 2 / 1	4 / 2 / 1	0.80	0.90
4	FoodDash	5 / 4 / 3	12 / 6 / 4	1.05	1.15
5	EduPortal	2 / 2 / 1	6 / 3 / 1	0.90	0.80
6	BankGuard	10 / 5 / 2	10 / 5 / 4	1.20	0.75
7	FitTrack	1 / 1 / 2	4 / 2 / 1	0.85	0.95
8	AutoPart	3 / 2 / 1	7 / 4 / 1	0.95	1.00
9	GreenGrid	6 / 3 / 1	9 / 3 / 2	1.15	0.85
10	Cinema	1 / 1 / 1	5 / 2 / 1	0.75	0.90
11	PetHotel	2 / 1 / 1	6 / 3 / 1	0.85	1.00
12	Warehouse	8 / 4 / 2	11 / 5 / 3	1.05	1.10
13	CryptoWallet	12 / 5 / 3	8 / 4 / 5	1.25	0.80
14	RealEstate	2 / 2 / 2	7 / 3 / 1	0.90	0.95
15	SmartHome	10 / 6 / 2	10 / 6 / 2	1.10	0.85
16	EventMaster	2 / 1 / 1	5 / 4 / 1	0.80	1.05
17	SteamCloud	15 / 8 / 4	15 / 8 / 4	1.15	1.10
18	LawCase	1 / 2 / 2	6 / 3 / 2	0.95	0.90
19	TaxiRide	4 / 3 / 2	9 / 4 / 1	1.00	1.05
20	Library	1 / 1 / 1	4 / 2 / 0	0.70	0.95

Крок 3. Управління невизначеністю за методом PERT

Мета: Навчитися прогнозувати терміни завершення проєкту в умовах невизначеності та оцінювати рівень часових ризиків.

Ваше завдання:

1) Розрахунок PERT: Використовуючи дані свого варіанту, обчисліть очікувану тривалість проєкту (E).

2) Оцінка ризику: Розрахуйте стандартне відхилення (SD). Чим вище це число, тим менш передбачуваним є ваш проєкт.

$$\text{Очікуваний час (E): } E = \frac{O + 4M + P}{6}$$

$$\text{Стандартне відхилення (SD): } SD = \frac{P - O}{6}$$

3) Визначення "Безпечного дедлайну": Розрахуйте дату релізу з імовірністю 95% за формулою: $E + 2 * SD$

4) Managerial Report:

- Яку дату (кількість днів) ви назвете замовнику як "гарантовану"? Чому не можна називати просто реалістичний час (M)?
- Аналіз розриву: Якщо різниця між O та P занадто велика, які саме ризики можуть до цього призвести?

Шпаргалка для "Управлінського звіту":

- Чому PERT кращий за M: Реалістична оцінка (M) має ймовірність успіху лише близько 50%. Для бізнесу це ризик "50/50". Метод PERT зміщує акцент на безпеку.
- Коефіцієнт ризику: Якщо $SD > 15$ проєкт вважається високо-ризиковим. Менеджеру варто закласти додатковий бюджет на випадок форс-мажорів.

Індивідуальні варіанти для Завдання 3 (Крок 3): Розрахунок PERT

№	Назва системи	Оптимістичний (O, дні)	Реалістичний (M, дні)	Песимістичний (P, дні)
1	SmartClinic	45	60	90
2	AeroCheck	60	85	150
3	EcoLogistics	30	40	70
4	FoodDash	90	120	210
5	EduPortal	40	55	80
6	BankGuard	70	100	180
7	FitTrack	25	35	60
8	AutoPart	50	65	110
9	GreenGrid	55	75	130
10	Cinema	20	30	55
11	PetHotel	35	45	75
12	Warehouse	80	110	190
13	CryptoWallet	65	90	160
14	RealEstate	45	55	100
15	SmartHome	75	95	165
16	EventMaster	30	42	85
17	SteamCloud	100	140	250
18	LawCase	40	60	110
19	TaxiRide	50	70	120
20	Library	25	32	55

Крок 4: Фінансове планування та формування кошторису (Commercial Proposal)

Мета: На основі попередніх розрахунків (UCP та PERT) визначити вартість розробки проєкту та сформулювати фінансове обґрунтування для замовника.

Ваше завдання:

1) Розрахунок бюджету на розробку:

- Візьміть загальну кількість людино-годин, яку ви розрахували на Кроці 2 (нагадаю: $UCP * 20$ люд-год).
- Прийміть середню ставку розробника для вашого проєкту: \$35 / год.
- Обчисліть фонд оплати праці:

$$\text{Budget} = \text{Total_Hours} \times 35.$$

2) Розрахунок "Резерву на ризики":

Використовуючи стандартне відхилення (SD) з Кроку 3, розрахуйте фінансовий буфер. Це сума, яку ви закладаєте на випадок затримок або непередбачуваних правок.

$$Risk_Buffer = Budget \times \left(\frac{SD}{E} \right).$$

Примітка: Якщо ризиковий бюджет перевищує 25%, студент мусить запропонувати заходи щодо зменшення невизначеності.

3) Формування фінальної пропозиції:

Заповніть підсумкову таблицю для замовника (замість N та M підставити свої значення):

Стаття витрат	Значення (Дні / Години)	Сума (\$)
Основна розробка	N люд-год	M \$
Резерв на ризики (Risk Buffer)	N % від бюджету	M \$
Гарантований термін релізу	N дні (PERT Safe)	—
ЗАГАЛЬНА ВАРТІСТЬ ПРОЄКТУ	—	M \$

4) Managerial Report (Висновок):

Дайте відповідь на питання:

- Яка частка бюджету припадає на ризики? (Якщо більше 20% — поясніть, чому проєкт такий нестабільний).
- Чому замовнику вигідніше прийняти вашу ціну з "буфером", ніж дешеву пропозицію конкурента, який обіцяє зробити все в оптимістичний термін (O)?

Критерії оцінювання:

- Математична точність: Правильність застосування формул UCP та PERT.
- Обґрунтування складності: Чому актор або сценарій отримали саме такий бал.
- Ризик-менеджмент: Розуміння того, як «сигма» у PERT захищає бюджет проєкту.

Порада студентам:

Бачите різницю в методах? Application Composition — це для "швидкого побачення" з клієнтом. UCP — це "шлюбний контракт". А PERT — це страховка вашого спокою. Використовуйте їх разом, щоб не працювати безкоштовно у вихідні/